



NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY

SCHOOL OF MECHANICAL AND MANUFACTURING
ENGINEERING

RAI-833 Deep Learning

Tutorial 16: Generative Adversarial Networks (GANs)

Adversarial Training for Handwritten Digit Synthesis

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

1 Objectives

The primary objective of this tutorial is to explore the theoretical and practical implementation of Generative Adversarial Networks (GANs). Key goals include:

- **Analyze the Minimax Game:** Understand the adversarial relationship where the Generator attempts to maximize the Discriminator's error, while the Discriminator attempts to minimize it.
- **Architecture Implementation:** Design and train custom Generator and Discriminator architectures using PyTorch.
- **Latent Space Exploration:** Visualize how random noise is transformed into meaningful data through learning a continuous manifold.
- **Performance Evaluation:** Utilize loss convergence and discriminator confidence metrics to evaluate generative stability.

2 Methodology

2.1 Adversarial Architecture

The GAN framework employed consists of two distinct sub-networks that engage in a synergistic competitive training process:

- **The Generator (G):** Acts as the generative agent. It receives a low-dimensional latent vector z (sampled from a normal distribution) and reconstructs it into a 28x28 pixel grayscale image mimicking the MNIST digit distribution. We utilized the Tanh activation function in the output layer to bound the synthesized pixel values within the interval $[-1, 1]$.
- **The Discriminator (D):** Acts as the critic. It is a binary classifier that maps an input image to a scalar probability $D(x) \in [0, 1]$, representing the likelihood that the input is a real digit from the dataset rather than a synthetic creation by G .

2.2 Mathematical Framework

The system is trained by solving a two-player minimax game defined by the value function $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (1)$$

3 Results and Discussion

3.1 Adversarial Convergence Analysis

The training process exhibited a non-stationary optimization behavior. As observed in Figure 1, the loss values for both models fluctuate, reflecting the adversarial dynamic where networks continuously adjust weights to counter improvements in their opponent.

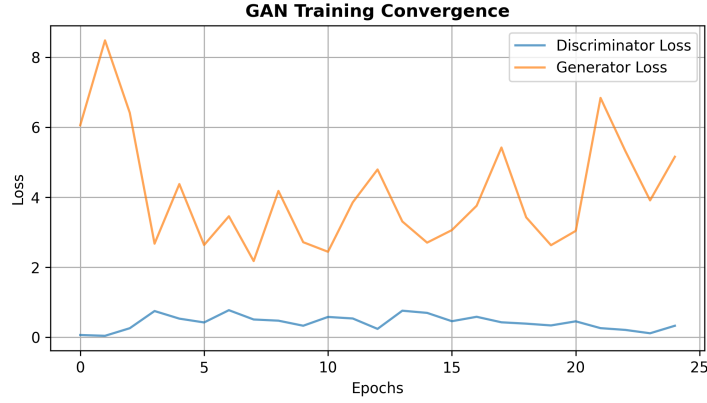


Figure 1: GAN Training Convergence: The alternating nature of the loss indicates competitive adversarial learning.

3.2 Generative Manifold and Latent Space

The Generator successfully synthesized high-fidelity digit representations (Figure 2). To assess whether the model had simply memorized the training data, we performed latent space interpolation (Figure 3). The visual transition between digits validates that the model has learned a continuous manifold where the network captures structural features.



Figure 3: Latent Space Morphing.

Figure 2: Final synthesized digits.

3.3 Discriminator Calibration

Figure 4 illustrates the Discriminator's confidence at the end of training. The overlap in the distribution of confidence scores for real and fake images suggests that the Generator has reached a high level of realism, successfully "fooling" the Discriminator.

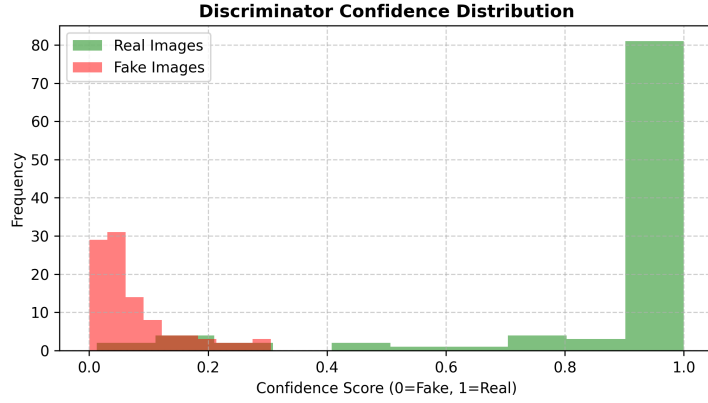


Figure 4: Discriminator Confidence Distribution: The overlapping distributions indicate high generative realism.

4 Conclusion

This tutorial successfully demonstrated the power of Generative Adversarial Networks in synthesizing realistic data distributions. Through the competitive adversarial training loop, the model learned to produce digits that are indistinguishable from real MNIST samples. The results emphasize that the stability of GANs relies heavily on the balanced competition between the Generator and Discriminator, offering a sophisticated approach to generative modeling.